Modul Mahasiswa Lab 08 - Cloud Notes dengan Next.js

Kebijakan kelas: Lab ini latihan formatif, tanpa tugas, nilai, atau penyerahan terpisah. Satu proyek besar dikerjakan oleh kelompok **3 orang**, dengan presentasi checkpoint minggu 7 (UTS) dan hasil akhir minggu 14 (UAS). Simpan hasil lab hanya bila berguna sebagai referensi atau bukti proses proyek. Baca brief proyek kelompok. Bobot resmi tetap mengikuti RPS/LMS.

Sesi RPS: 8 · **Mode utama:** lokal · **Bukti latihan opsional untuk proyek:** source, laporan, screenshot hasil sendiri, dan commit Git.

Tujuan dan konsep

Anda akan menjalankan frontend Next.js App Router untuk Cloud Notes. Halaman (`src/app/page.tsx`) memakai state dan form di browser; Route Handler (`src/app/api/notes/route.ts`) meneruskan GET/POST ke API Compose Lab 06 dari server Next.js. Mode Supabase memakai Auth dan RLS jika akun tersedia. Setelah lab, Anda harus dapat menjelaskan perbedaan Client Component, Route Handler server, API lokal, dan variabel yang boleh terlihat di browser.

Contoh Todo pada slide sesi 8 menjelaskan pola Next.js. Praktik yang dapat dijalankan dari repositori ini memakai Cloud Notes dan path `src/app/api/notes/route.ts`.

Jenis bukti: gambar Docker Desktop dan browser adalah screenshot antarmuka aktual; kartu terminal menata ulang output perintah yang benar-benar dijalankan dan memiliki file `.txt` pendamping. Kartu itu bukan screenshot jendela terminal mentah. Ambil screenshot mesin Anda sendiri untuk laporan.

Persiapan

- Node.js **minimal 22.0.0** untuk versi paket pada `package-lock.json` repo ini, Docker dan Compose, serta repo Lab 06 yang sehat pada `http://127.0.0.1:18006/health`. Next.js sendiri menerima Node 20.9+, tetapi dependensi `@supabase/supabase-js` yang terkunci di proyek ini mensyaratkan Node 22+; ikuti kebutuhan seluruh proyek dan dukungan runtime Supabase.
- Port 3000 belum dipakai proses lain. Hentikan server Lab 01/03 bila masih hidup.
- Dari root repo Lab 06, siapkan `.env` dari `.env.example` sesuai README revisi lokal dan jalankan `docker compose up --build -d --wait`. File secret terpisah tidak dipakai pada revisi ini. Jangan commit file `.env` atau password nyata.

Port API revisi lokal: `LOCAL_API_URL` pada `.env.local` Lab 08 memakai `http://127.0.0.1:18006`. Sesuaikan file konfigurasi yang sudah ada, lalu hentikan dan jalankan kembali server Next.js. Screenshot acuan yang menampilkan API pada port 8000 berasal dari pengujian versi sebelumnya; gunakan port 18006 untuk langkah dan pemeriksaan sekarang.



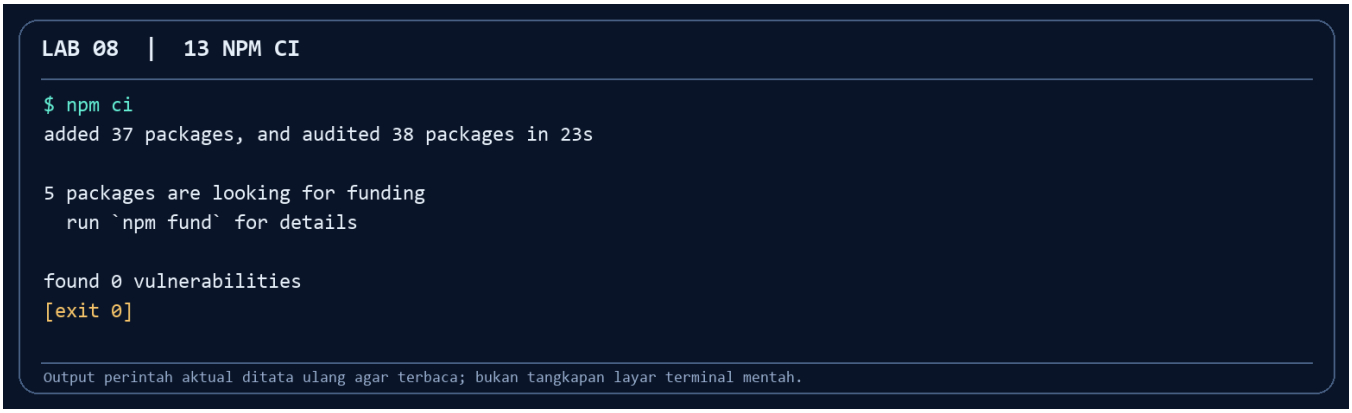
Langkah: clone `meet6CloudService` dan `meet8CloudService` pada satu komputer/Codespace, lalu jalankan `git remote -v` di masing-masing folder. Fungsi: memastikan Next dapat memanggil API pada `localhost:18006`. Cara kerja: kedua proses memakai jaringan host yang sama; dua Codespace terpisah memiliki `localhost` berbeda. Baca hasil: remote Lab 06 dan Lab 08 menuju repo yang berbeda, dengan folder kerja berdampingan. Output lengkap historis.

Dari root repo Lab 08, jalankan PowerShell:

```
node --version
npm --version
npm ci
Copy-Item .\.env.example .\.env.local
npm run dev
```

Pada Bash/WSL, gunakan `cp .env.example .env.local`. Jika `.env.local` sudah ada, periksa isinya sebelum menyalin ulang. Buka `http://localhost:3000` dan `http://localhost:3000/api/health`.

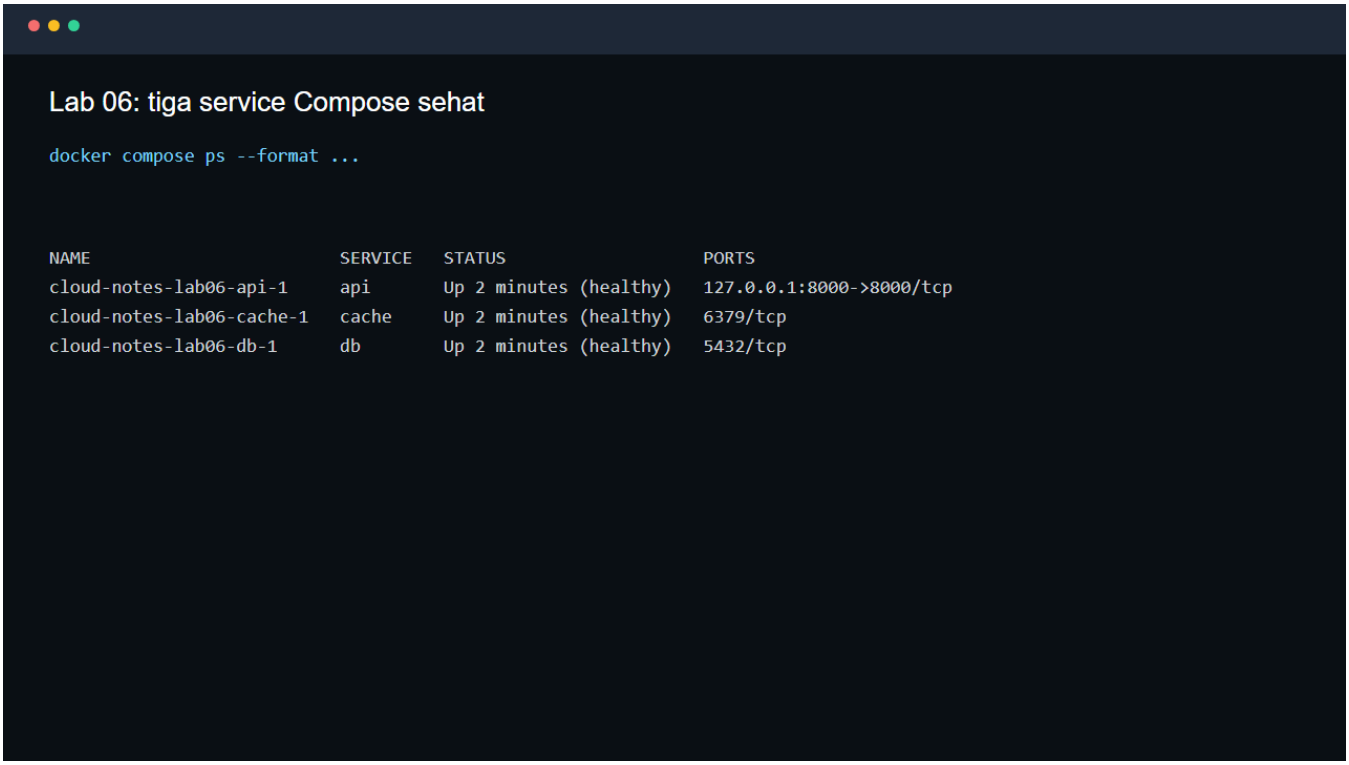
Checkpoint runtime: `node --version` harus menunjukkan `v22.0.0` atau lebih baru dan `npm --version` harus menampilkan nomor versi. Jika Node masih 20.x atau muncul `EBADENGINE` ketika `npm ci`, siapkan Node 22+ pada lingkungan praktik, buka kembali terminal, lalu ulangi kedua pemeriksaan sebelum instalasi paket.



Langkah: jalankan `npm ci` dari repo Lab 08 sebelum `npm run dev`. Fungsi: memasang versi dependensi yang terkunci di `package-lock.json`. Cara kerja: npm menyusun ulang `node_modules` dan memeriksa paket. Baca hasil: 37 paket terpasang, exit 0; jumlah paket/versi dapat berubah jika lockfile berubah. Jika Windows menampilkan `EPERM/ENOTEMPTY`, hentikan proses Next yang masih memakai `node_modules`, lalu ulangi. Gambar adalah render dari output run aktual.



Langkah: periksa `node --version`, `git check-ignore -v .env.local`, `/api/health`, dan `/api/ready` setelah dua server hidup. Fungsi: membuktikan prasyarat runtime dan dependensi sebelum membuat catatan. Cara kerja: health Next memeriksa prosesnya sendiri; ready memanggil health API Lab 06. Baca hasil: Node memenuhi minimum, `.env.local` diabaikan Git, kedua endpoint HTTP 200 saat API/DB/cache sehat. Output lengkap.



Langkah: dari repo Lab 06 jalankan `docker compose up --build -d --wait` dan `docker compose ps`. Fungsi: menyediakan API, PostgreSQL, dan Redis. Cara kerja: Compose menunggu healthcheck. Screenshot historis memakai host port 8000; revisi lokal sekarang memakai 18006. Baca hasil: tiga service sehat; hanya lanjutkan ketika `/health` Lab 06 merespons.

Praktik bertahap

1. **Kenali komponen.** Buka `src/app/page.tsx` dan `src/app/api/notes/route.ts`. Tandai `'use client'`, event form, `fetch('/api/notes')`, serta `LOCAL_API_URL` yang hanya dibaca server. Untuk menemukan barisnya cepat, jalankan `rg -n 'use client|fetch|LOCAL_API_URL|export async function' src/app/page.tsx src/app/api/notes/route.ts` dari root repo (atau `Select-String` pada PowerShell jika `rg` tidak terpasang). Catat aliran browser → Next Route Handler → API Lab 06 → PostgreSQL.
2. **Uji pembacaan.** Buka halaman dan cek label `Mode aktif: API lokal`. Refresh daftar. Dari terminal kedua, jalankan `Invoke-RestMethod http://localhost:3000/api/health` (PowerShell) atau `curl -fsS http://localhost:3000/api/health` (Bash). Harapkan HTTP 200.
3. **Tulis catatan.** Isi judul dan isi, klik **Simpan**, lalu pastikan catatan tampak pada daftar. Cocokkan dengan `http://127.0.0.1:18006/notes`. Amati terminal Next.js: log JSON `notes_proxy` memuat `requestId`, status, dan durasi tanpa isi catatan.
4. **Uji validasi.** Kirim body tanpa judul ke `/api/notes`; Route Handler seharusnya mengembalikan HTTP 400. Di PowerShell, jalankan perintah berikut dari terminal kedua:

```
try { Invoke-RestMethod http://localhost:3000/api/notes -Method Post -ContentType application/json -Body (@{content='uji'} | ConvertTo-Json) } catch { [int]$_.Exception.Response.StatusCode }
```

Di Bash, gunakan `curl -i http://localhost:3000/api/notes -H 'Content-Type: application/json' -d '{"content":"uji"}'`. Jangan mengirim password/token dalam request contoh. 5. **Verifikasi build.** Hentikan `npm run dev` dengan `Ctrl+C`, kemudian jalankan `npm run typecheck` dan `npm run build`. Catat hasilnya. Saat build selesai, jalankan kembali `npm run dev` jika perlu screenshot.

Tampilan pada setiap langkah

Langkah 1 - jalur source. Output pencarian ini berasal dari file source repo yang sama dengan yang dijalankan.

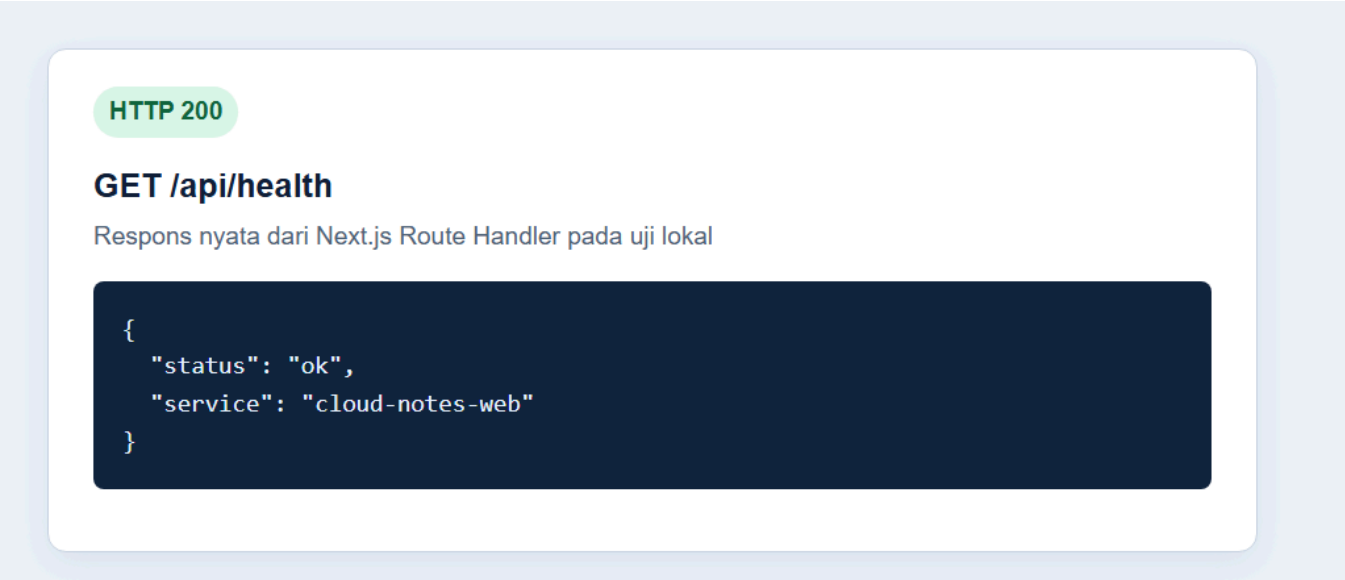
```
LAB 08  |  15 SOURCE TOUR

$ rg -n 'use client|fetch|LOCAL_API_URL|export async function' src/app/page.tsx
src/app/api/notes/route.ts
src/app/api/notes/route.ts:4:const baseUrl = process.env.LOCAL_API_URL ||
'http://127.0.0.1:8000'
src/app/api/notes/route.ts:10:    const upstream = await fetch(`${baseUrl}/notes`, {
src/app/api/notes/route.ts:32:export async function GET() {
src/app/api/notes/route.ts:36:export async function POST(request: Request) {
src/app/page.tsx:1:'use client'
src/app/page.tsx:27:      const response = await fetch('/api/notes', { cache: 'no-store'
 })
src/app/page.tsx:58:        const response = await fetch('/api/notes', { method: 'POST',
 headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ title, content
}) })
[exit 0]

Output perintah aktual ditata ulang agar terbaca; bukan tangkapan layar terminal mentah.
```

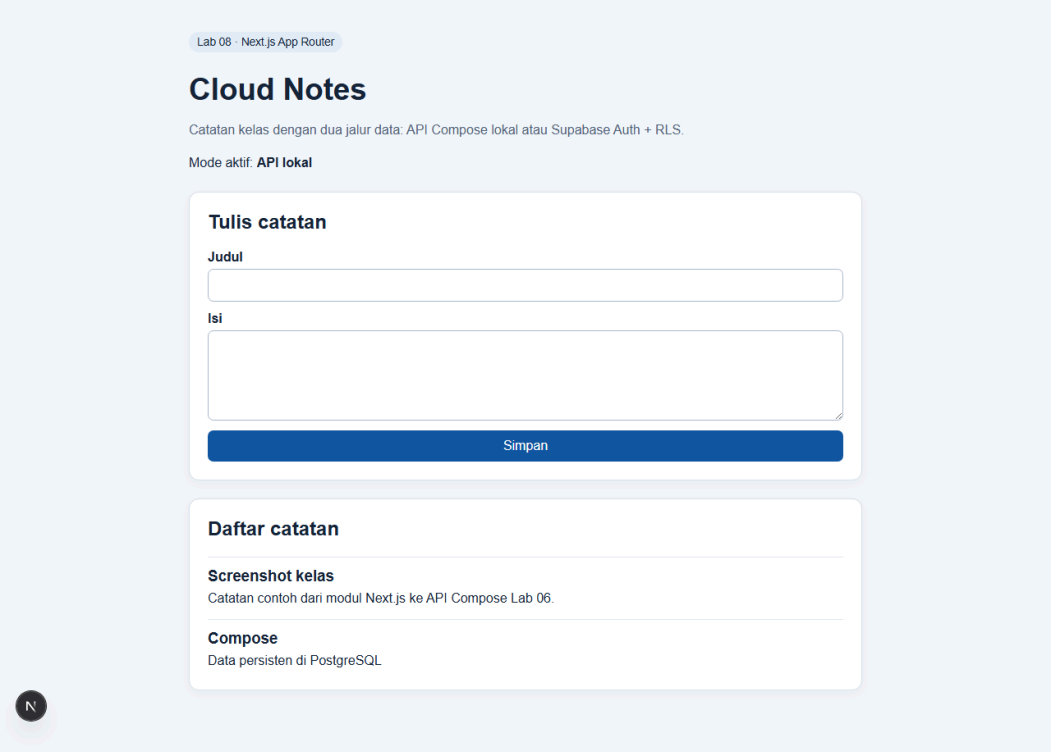
- **Langkah:** Jalankan perintah `rg` pada langkah 1. **Fungsi:** menemukan batas browser/server tanpa menebak nama file. **Cara kerja:** `rg` mencari pola pada `page.tsx` dan Route Handler. **Baca hasil:** `'use client'` serta `fetch('/api/notes')` berada di halaman; `LOCAL_API_URL` dan GET/POST berada di server. Output lengkap.

Langkah 2 - endpoint health Next.js. Kartu berikut menyusun ulang respons HTTP 200 dan JSON dari server lokal agar mudah dibaca; tampilan bawaan browser/terminal Anda dapat berbeda. Cek health API Lab 06 secara terpisah.



- **Langkah:** Jalankan `npm run dev` dari root repo Lab 08, lalu buka `http://localhost:3000/api/health`. **Fungsi:** Memeriksa Route Handler Next.js sebelum membuat catatan. **Cara kerja:** Next.js memproses GET pada server dan mengirim status/JSON; pemeriksaan API Lab 06 dilakukan terpisah. **Baca hasil:** Baca HTTP 200 dan JSON health; jangan menyimpulkan backend DB sehat hanya dari halaman ini.

Langkah 3 - form dan catatan di web. Setelah tombol **Simpan**, catatan terlihat pada daftar. Screenshot contoh ini berasal dari praktik lokal; simpan hasil Anda sendiri bila berguna untuk proyek.



- **Langkah:** Buka `http://localhost:3000`, isi judul/konten, lalu klik **Simpan**. **Fungsi:** Menguji frontend Cloud Notes dan daftar catatan. **Cara kerja:** Browser mengirim request ke Route Handler Next.js; server meneruskan ke API Lab 06 yang menyimpan ke PostgreSQL. **Baca hasil:** Cari mode API lokal, pesan sukses, dan catatan baru pada daftar setelah refresh.

LAB 08 | 16 POST REQUEST ID

```
$ $r = Invoke-WebRequest http://localhost:3000/api/notes -Method Post -ContentType
application/json -Body '{"title":"Audit bukti Lab08","content":"Jalur empat lapisan
berhasil"}' -UseBasicParsing; 'POST /api/notes: HTTP ' + $r.StatusCode; 'X-Request-ID: '
 + $r.Headers['X-Request-ID']; $r.Content
POST /api/notes: HTTP 201
X-Request-ID: 783d48a8-67e0-4c6d-90ab-ed708448df11
{"id":6,"title":"Audit bukti Lab08","content":"Jalur empat lapisan
berhasil","created_at":"2026-10-07T15:11:43.844136+00:00"}
[exit 0]
$ $r = Invoke-WebRequest http://localhost:3000/api/notes -UseBasicParsing; 'GET
/api/notes: HTTP ' + $r.StatusCode; if ($r.Content -match 'Audit bukti Lab08') { 'Judul
Audit bukti Lab08: ditemukan dalam respons' } else { 'Judul: TIDAK ditemukan' }
GET /api/notes: HTTP 200
Judul Audit bukti Lab08: ditemukan dalam respons
[exit 0]
```

Output perintah aktual ditata ulang agar terbaca; bukan tangkapan layar terminal mentah.

- **Langkah:** dari terminal kedua kirim POST `http://localhost:3000/api/notes` berisi `title` dan `content`, lalu GET URL yang sama. **Fungsi:** memeriksa jalur API tanpa UI serta menghubungkan request dengan log. **Cara kerja:** Next memvalidasi dan meneruskan POST ke Lab 06; header `X-Request-ID` ikut dalam respons. **Baca hasil:** POST HTTP 201, GET HTTP 200, judul tersimpan. Pada run ini header ID `783d48a8-67e0-4c6d-90ab-ed708448df11` cocok dengan log Next `notes_proxy` berstatus 201; log tidak memuat isi catatan. Perintah dan output lengkap.

Langkah 4 - validasi. Request tanpa judul dan JSON rusak sama-sama menghasilkan HTTP 400 dari Route Handler.

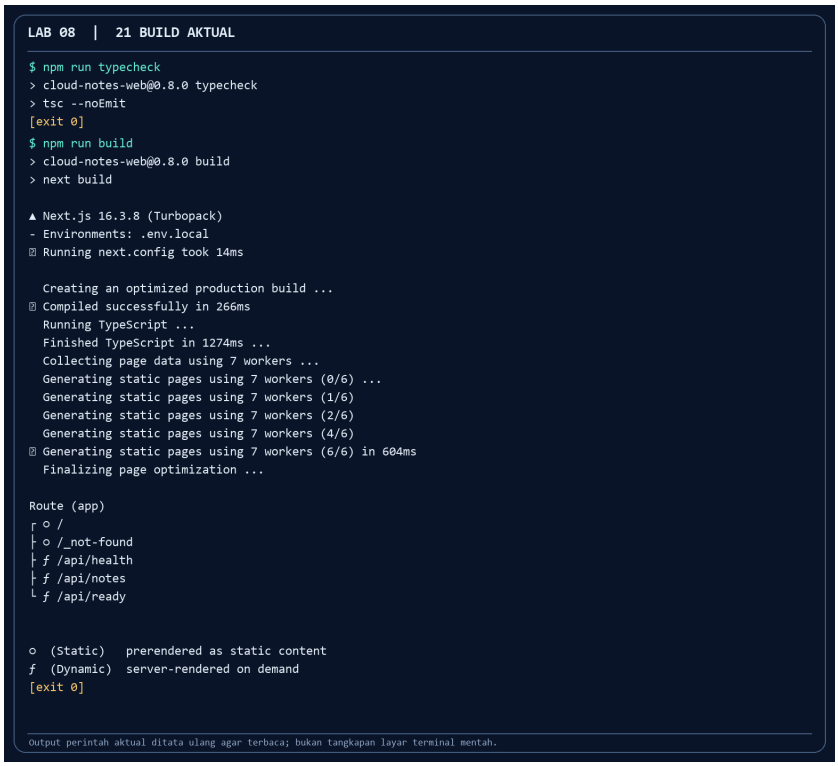
LAB 08 | 17 VALIDASI AKTUAL

```
$ try { Invoke-WebRequest http://localhost:3000/api/notes -Method Post -ContentType
application/json -Body '{"content":"uji"}' -UseBasicParsing } catch { 'POST tanpa title:
 HTTP ' + [int]$_.Exception.Response.StatusCode; $stream =
$_.Exception.Response.GetResponseStream(); if ($stream) { (New-Object
System.IO.StreamReader($stream)).ReadToEnd() } }
POST tanpa title: HTTP 400
{"error":"Judul dan isi wajib diisi"}
[exit 0]
$ try { Invoke-WebRequest http://localhost:3000/api/notes -Method Post -ContentType
application/json -Body '{' -UseBasicParsing } catch { 'POST JSON rusak: HTTP ' +
[int]$_.Exception.Response.StatusCode; $stream =
$_.Exception.Response.GetResponseStream(); if ($stream) { (New-Object
System.IO.StreamReader($stream)).ReadToEnd() } }
POST JSON rusak: HTTP 400
{"error":"JSON tidak valid"}
[exit 0]
```

Output perintah aktual ditata ulang agar terbaca; bukan tangkapan layar terminal mentah.

- **Langkah:** Kirim POST `/api/notes` tanpa judul lalu ulangi dengan body { melalui terminal. **Fungsi:** memastikan input tidak valid ditolak sebelum disimpan. **Cara kerja:** Route Handler memvalidasi payload dan mengembalikan JSON error tanpa membuat catatan baru. **Baca hasil:** keduanya HTTP 400, masing-masing dengan pesan yang sesuai. Perintah dan output lengkap.

Langkah 5 - pemeriksaan build. TypeScript dan build Next.js lulus sebelum source dipush.



- **Langkah:** Jalankan `npm run typecheck` dan `npm run build` dari root repo Lab 08 sesudah menghentikan `npm run dev`. **Fungsi:** memeriksa tipe TypeScript dan kelayakan build sebelum push. **Cara kerja:** compiler memeriksa tipe, lalu Next.js membuat bundle produksi. **Baca hasil:** keduanya exit 0; route `/api/health`, `/api/ready`, dan `/api/notes` tercantum. Output lengkap.

Jalur cloud opsional: ikuti README Lab 08 untuk menjalankan `schema.sql` Lab 07 di Supabase, memasukkan URL dan *publishable key* pada `.env.local`, menguji dua akun, lalu mengimpor root repo Lab 08 ke Vercel dari repo Git. Jangan memakai `LOCAL_API_URL=localhost` pada deployment publik. `NEXT_PUBLIC_` hanya untuk nilai yang memang boleh dilihat browser; jangan menaruh service role/secret key di sana.

Paket ini belum berisi screenshot hasil deploy Supabase/Vercel karena jalur cloud memerlukan proyek dan akun peserta. Jika mengerjakannya, simpan screenshot project setting yang tidak menampilkan key rahasia, URL deploy, serta uji dua akun dari proyek Anda sendiri.

Pertanyaan untuk laporan

1. Mengapa `page.tsx` memerlukan `'use client'`, tetapi Route Handler berjalan di server?
2. Apa yang terjadi jika API Lab 06 dimatikan? Catat status/respons dan bagian sistem yang masih hidup.
3. Mengapa URL dan publishable key Supabase boleh berada pada `NEXT_PUBLIC_`, sedangkan service role key tidak?
4. Untuk halaman daftar catatan yang berubah setiap pengguna, kapan SSR/SSG/ISR relevan? Jelaskan pilihan Anda.

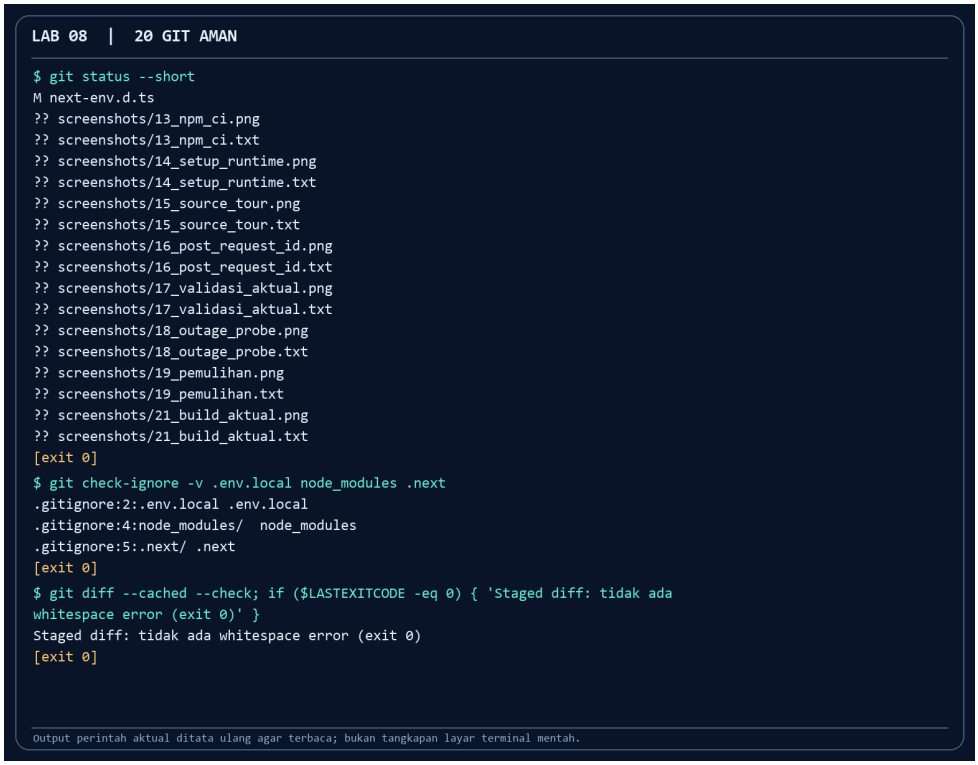
Bukti dan Git

Salin `TEMPLATE_LAPORAN.md` menjadi `hasil/lab08.md` dari root repo. Sertakan screenshot **hasil Anda sendiri** (form + daftar), respons health, hasil typecheck/build, contoh log JSON yang aman, diagram aliran request, dan jawaban pertanyaan. Jika jalur cloud dipakai, tambahkan URL preview/produksi dan uji RLS dua akun.

Dari root repo Lab 08:

```
git status --short
git add .
git diff --cached --name-only
git diff --cached --check
git commit -m "lab08: frontend notes dan route handler"
git push
```

Simpan bukti milik Anda di `hasil/bukti/` bila ada; laporan tetap perlu memuat bukti hasil sendiri. Periksa daftar staged: `.env.local`, `node_modules`, `.next`, dan kunci rahasia tidak boleh masuk. Baca panduan Git jika repo belum punya remote.



Langkah: jalankan `git status --short`, `git check-ignore -v .env.local node_modules .next`, lalu `git diff --cached --check`. Fungsi: memastikan hasil praktik yang akan dipush tidak membawa konfigurasi lokal atau build. Cara kerja: Git menampilkan perubahan, mencocokkan aturan ignore, dan memeriksa diff staged. Baca hasil: ketiga path lokal diabaikan; pada cuplikan ini file praktik belum staged, jadi ulangi pemeriksaan setelah `git add`. Screenshot push terakhir harus berasal dari repo Anda sendiri. Output lengkap.

Selesai dan kendala umum

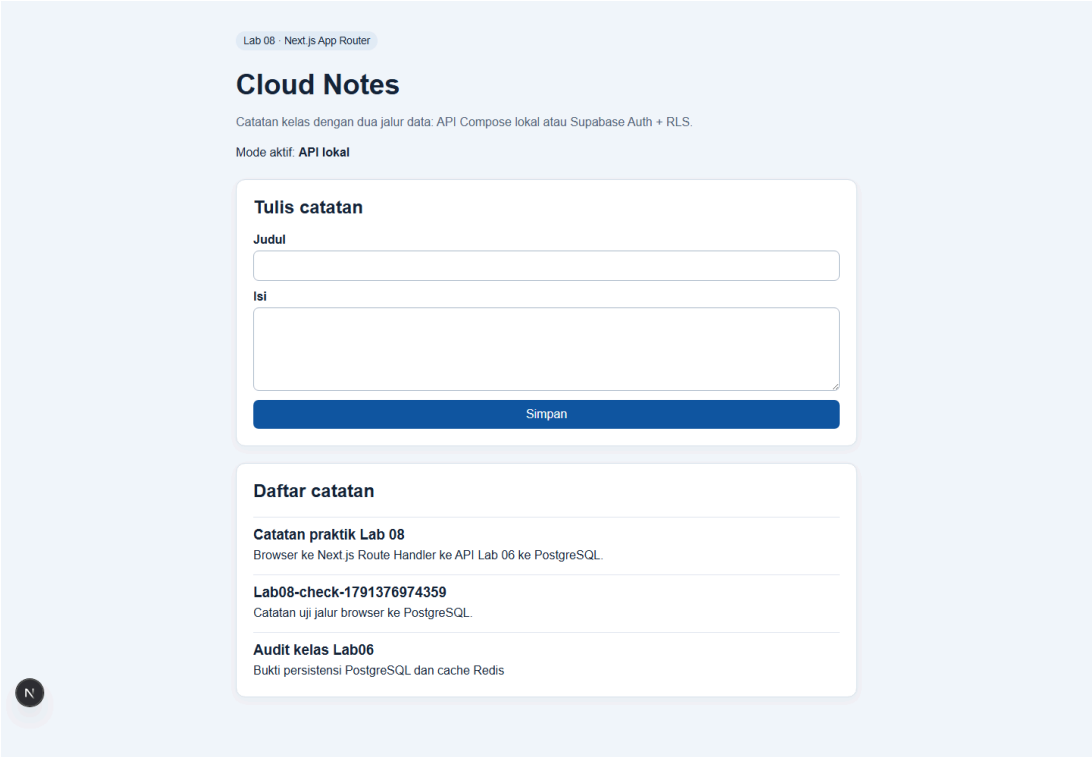
Hentikan Next dengan `Ctrl+C`. Setelah tidak dipakai lab lain, dari root repo Lab 06 jalankan `docker compose --profile debug down`; volume tetap ada. Jika halaman menampilkan API lokal gagal, periksa health Lab 06 dan `LOCAL_API_URL`. Jika port 3000 terpakai, hentikan Lab 01/03 atau proses Next lama. Jika build gagal, cek versi Node dan kesalahan TypeScript yang ditunjuk terminal.

Pada Windows, `npm ci` dapat gagal dengan `EPERM/ENOTEMPTY` jika proses Next lama masih memakai file native di `node_modules`. Hentikan proses `npm run dev` beserta child Next yang memang milik repo ini, lalu ulangi `npm ci`; dalam uji lokal retry berhasil dan build berikutnya exit 0.

Challenge kerja sehari-hari: catatan tidak tersimpan - kunci lengkap

Kasus: staf menekan **Simpan** tetapi daftar catatan tidak berubah. Tugas Anda menelusuri browser → Next.js → API Lab 06 → PostgreSQL, membedakan input salah dari layanan mati, lalu menunjukkan bukti pulih. Lab 06 berada di repo terpisah SeedFlora/meet6CloudService. Jalankan Lab 06 dan Lab 08 pada **komputer/Codespace yang sama**: `localhost` di dua Codespace berbeda tidak saling terhubung. Buka terminal pertama di clone Lab 06 untuk `docker compose up --build -d --wait`, lalu terminal kedua di root repo Lab 08 untuk `npm run dev`.

1. **Buktikan lapisan hidup.** Di terminal ketiga jalankan `curl -i http://localhost:3000/api/health`, `curl -i http://localhost:3000/api/ready`, dan `curl -i http://127.0.0.1:18006/health` (Bash). Di PowerShell gunakan `Invoke-WebRequest` untuk URL yang sama. `/api/health` memeriksa Next saja; `/api/ready` memanggil health API Lab 06 yang juga memeriksa PostgreSQL/Redis. Saat semua sehat, ketiganya HTTP 200. Bila browser membuka mode dev lokal, gunakan `http://localhost:3000`; endpoint dari terminal boleh memakai `127.0.0.1`.
2. **Bedakan validasi dari kegagalan backend.** PowerShell: `try { Invoke-RestMethod http://localhost:3000/api/notes -Method Post -ContentType application/json -Body '{"content":"uji"}' } catch { [int]$_.Exception.Response.StatusCode }`. Bash: `curl -i http://localhost:3000/api/notes -H 'Content-Type: application/json' -d '{"content":"uji"}'`. Kunci: HTTP **400** karena judul hilang; tidak ada catatan baru. JSON rusak juga HTTP 400. Ini terjadi sebelum request diteruskan ke API Lab 06.
3. **Simpan dan telusuri request.** Isi formulir browser dengan judul unik, misalnya `Audit catatan kelas`, lalu klik **Simpan**. Catatan muncul di daftar. Kunci terminal: `curl -i http://localhost:3000/api/notes` menghasilkan HTTP 200 dan judul itu; `curl -i -X POST http://localhost:3000/api/notes -H 'Content-Type: application/json' -d '{"title":"Audit API","content":"Jalur lengkap berhasil"}'` menghasilkan HTTP 201 serta header `X-Request-ID`. Cocokkan nilai header itu dengan `requestId` pada baris `notes_proxy` di terminal Next. Pada PowerShell, gunakan `Invoke-WebRequest`/`Invoke-RestMethod` dengan body JSON setara.
4. **Simulasikan outage secara terkontrol.** Di terminal Lab 06 jalankan `docker compose stop api` (biarkan DB/Redis hidup). `/api/health` Next tetap HTTP 200, `/api/ready` HTTP **503**, `/api/notes` HTTP **503**, dan halaman menampilkan pesan API lokal gagal. Terminal Next mencatat `notes_proxy_unavailable` tanpa isi catatan/password. Pulihkan dengan `docker compose start api`, tunggu `docker compose ps` menunjukkan `healthy`, lalu ulangi `/api/ready` dan daftar catatan; keduanya kembali berhasil. Hanya lakukan simulasi ini pada lingkungan praktik Anda sendiri.
5. **Verifikasi otomatis dan build.** Saat API sudah pulih jalankan `npm run check:local`: kunci **9 PASS, 0 FAIL**. Skrip membuat satu catatan uji bertanda `Lab08-check-...` agar membuktikan penulisan/pembacaan nyata. Hentikan `npm run dev` dengan `Ctrl+C`, lalu `npm run typecheck` dan `npm run build`; keduanya harus exit code 0. Setelah itu `npm run dev` lagi bila perlu screenshot.



Perintah/tindakan: `npm run dev`, buka `http://localhost:3000`, isi form dan klik **Simpan**. *Fungsi:* membuktikan aliran web sampai database. *Cara kerja:* browser POST ke Route Handler, server meneruskan ke API Lab 06, API menyimpan ke PostgreSQL, lalu web melakukan GET daftar. *Baca hasil:* judul baru terlihat di bagian Daftar catatan.

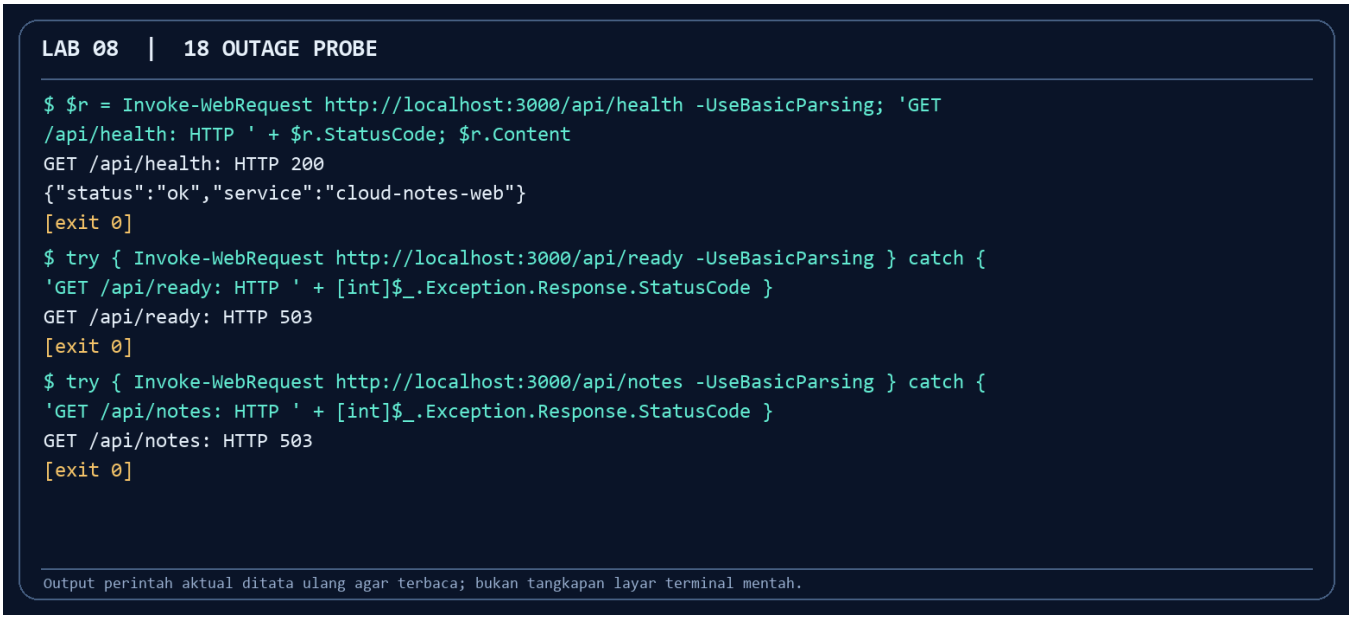


Perintah/tindakan: buka `http://localhost:3000/api/ready`. *Fungsi:* membedakan proses Next hidup dari dependensi Lab 06 siap. *Cara kerja:* Route Handler server meminta `/health` API Lab 06 dengan batas waktu tiga detik. *Baca hasil:* JSON `status: ready` dan HTTP 200 ketika API, PostgreSQL, serta Redis sehat; saat dependensi gagal, status 503.

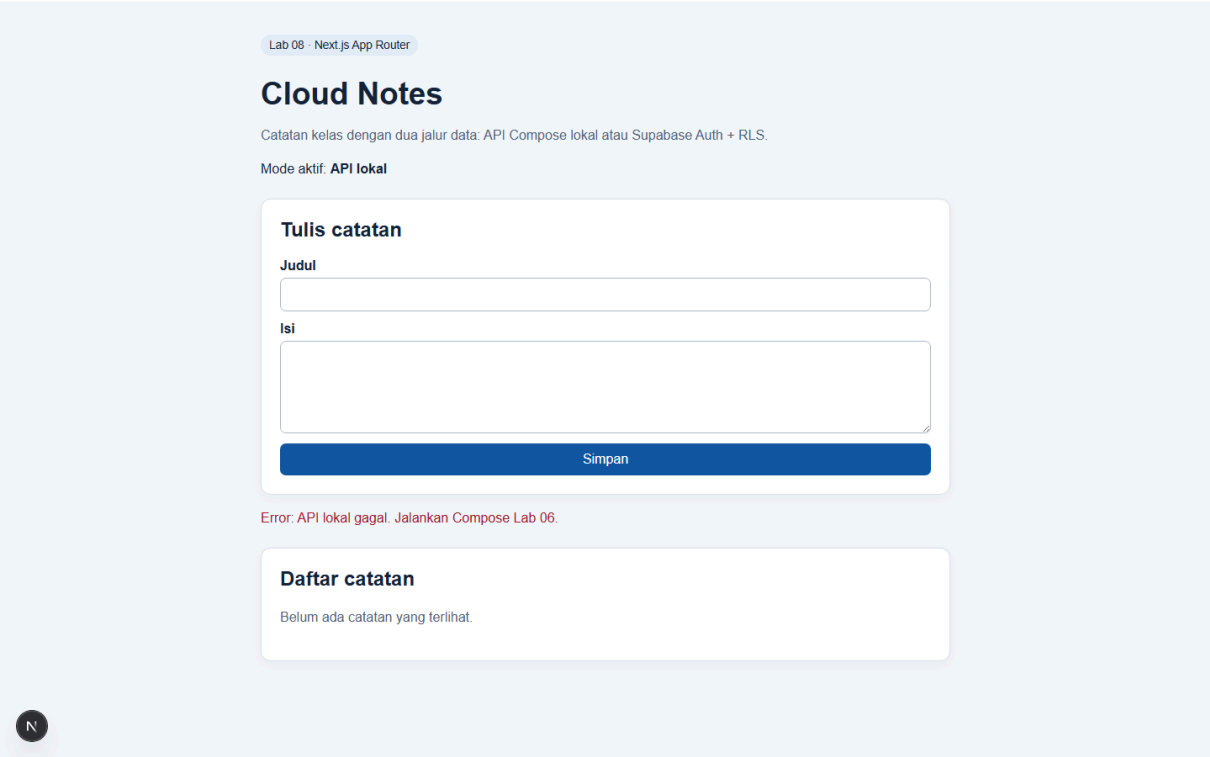
Pretty-print

{"status":"unavailable","dependency":"lab06-api"}

Perintah/tindakan: `docker compose stop api` di repo Lab 06 lalu buka kembali `/api/ready`. *Fungsi:* membuktikan pemantauan mendeteksi dependensi tidak siap. *Cara kerja:* request server Next ke health API gagal, lalu Route Handler memberi HTTP 503. *Baca hasil:* `status: unavailable`; `/api/health` Next tetap 200. Pulihkan dengan `docker compose start api` dan tunggu status healthy.



Langkah: setelah `docker compose stop api` dari Lab 06, jalankan `Invoke-WebRequest` untuk `/api/health`, `/api/ready`, dan `/api/notes` pada Next. Fungsi: menemukan lapisan yang gagal. Cara kerja: health Next berdiri sendiri, sedangkan ready dan notes memanggil API Lab 06. Baca hasil: HTTP 200/503/503. Perintah dan output lengkap.

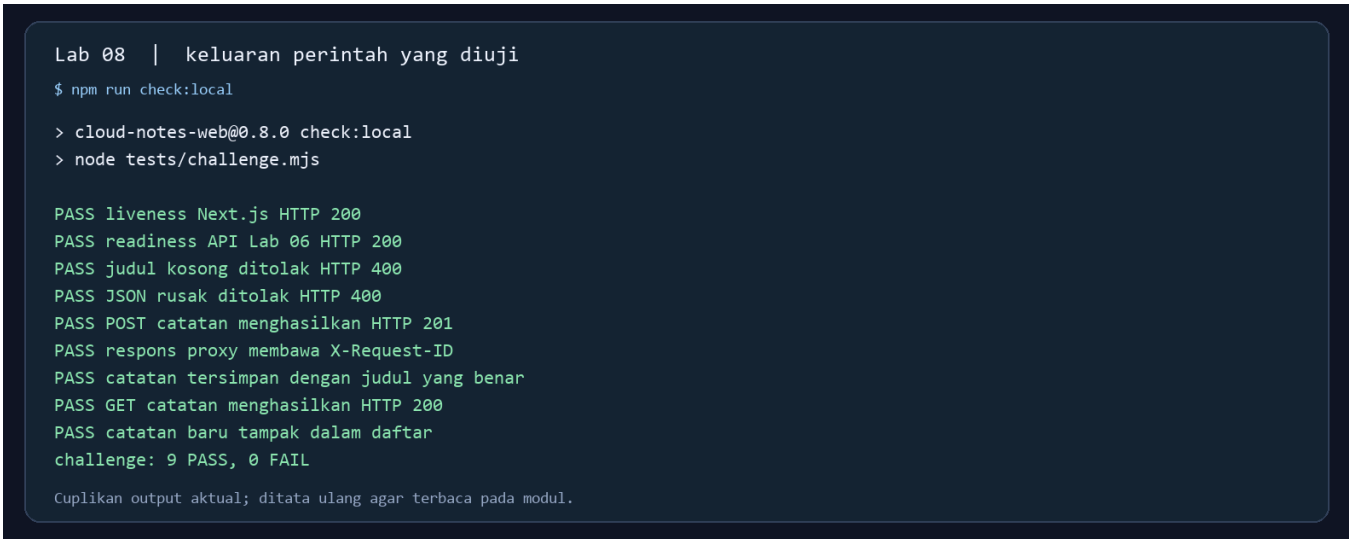


Perintah/tindakan: refresh halaman Cloud Notes selama API mati. *Fungsi:* melihat gejala yang dialami pengguna. *Cara kerja:* browser meminta `/api/notes`, proxy mengembalikan 503, UI menampilkan pesan error. *Baca hasil:* form tetap terlihat, daftar tidak terisi sampai API pulih; catatan di PostgreSQL tidak terhapus.

```
LAB 08  |  19 PEMULIHAN
$ $r = Invoke-WebRequest http://localhost:3000/api/ready -UseBasicParsing; 'GET
/api/ready: HTTP ' + $r.StatusCode; $r.Content
GET /api/ready: HTTP 200
{"status":"ready","dependency":"lab06-api"}
[exit 0]
$ $r = Invoke-WebRequest http://localhost:3000/api/notes -UseBasicParsing; 'GET
/api/notes: HTTP ' + $r.StatusCode; if ($r.Content -match 'Audit bukti Lab08') {
'Catatan lama: tetap ada setelah API hidup kembali' } else { 'Catatan lama: TIDAK
ditemukan' }
GET /api/notes: HTTP 200
Catatan lama: tetap ada setelah API hidup kembali
[exit 0]

Output perintah aktual ditata ulang agar terbaca; bukan tangkapan layar terminal mentah.
```

Langkah: jalankan `docker compose start api` dari Lab 06, tunggu `docker compose ps` menunjukkan `healthy`, lalu ulangi GET ready dan notes dari Lab 08. Fungsi: memastikan pemulihan end to end. Cara kerja: Next kembali dapat memanggil API yang tersambung ke DB/cache. Baca hasil: kedua GET HTTP 200 dan catatan sebelum outage masih ada. Perintah dan output lengkap.

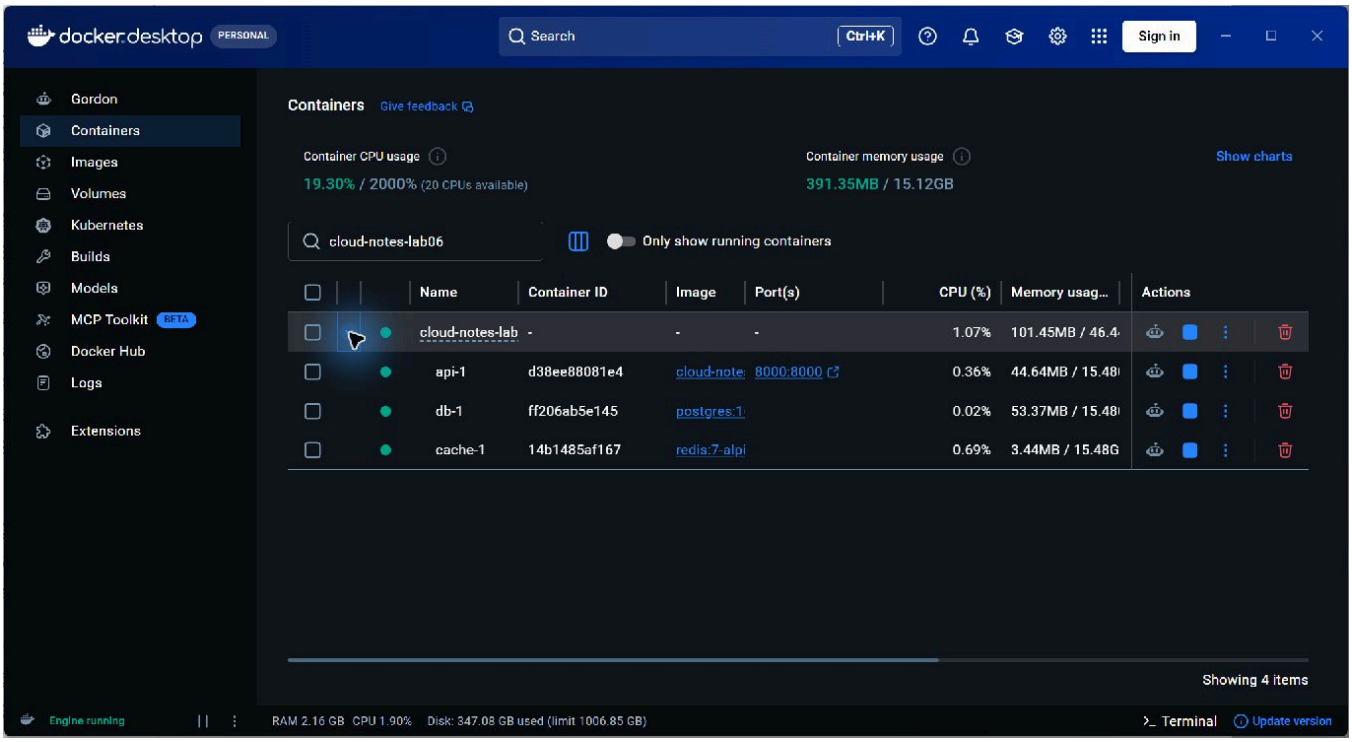


Perintah/tindakan: `npm run check:local` setelah API pulih. *Fungsi:* memeriksa kembali liveness, readiness, validasi, simpan, dan baca. *Cara kerja:* skrip Node memanggil Route Handler dan memastikan catatan uji baru muncul di GET. *Baca hasil:* `9 PASS, 0 FAIL`; gambar menata ulang keluaran perintah aktual agar terbaca.

Jawaban pertanyaan laporan

1. `page.tsx` memakai state, form, dan event di browser sehingga memerlukan `'use client'`. Route Handler menerima HTTP pada server Next; `LOCAL_API_URL` dibaca di server dan tidak perlu diekspos sebagai `NEXT_PUBLIC_`.
2. Jika API Lab 06 mati, halaman Next dan `/api/health` masih dapat merespons, tetapi `/api/ready` serta proxy `/api/notes` mengembalikan 503. Data lama di PostgreSQL tidak hilang; setelah API pulih, daftar muncul kembali.
3. `NEXT_PUBLIC_` dibundel ke browser. URL Supabase dan publishable key dirancang untuk client dengan RLS; service role/secret key memberi hak lebih luas dan harus tetap di server/secret store. Uji RLS dua akun pada jalur cloud bila tersedia.
4. Daftar catatan per pengguna tidak cocok dijadikan halaman statis bersama. Mode lokal ini memakai Client Component dan request tanpa cache. SSR bisa dipakai bila auth dan data diambil per request pada server; SSG/ISR hanya cocok untuk konten publik yang aman dibagikan dan tidak berubah per pengguna.

Bukti latihan yang boleh disimpan untuk proyek: screenshot browser milik Anda sebelum/sesudah simpan, status 400 dan 503, hasil pulih 200, satu `X-Request-ID` yang cocok dengan log, 9 PASS, build sukses, dan jawaban alur request. Nilai contoh pada screenshot di atas berasal dari uji lokal dan dapat berbeda pada mesin Anda.



Perintah/tindakan: dari repo Lab 06 jalankan `docker compose up --build -d --wait`, kemudian buka Docker Desktop. *Fungsi:* memastikan backend tiga service siap sebelum Next. *Cara kerja:* Compose menjalankan API, PostgreSQL, dan Redis pada network internal; Screenshot historis menampilkan host port 8000; revisi lokal sekarang memetakan API ke 18006. *Baca hasil:* ketiganya hijau/healthy.